Mobile Money Investigations

PostgreSQL and Neo4j on 20 million September 2026 transfers

Big Data Analytics | Project 2 | Group 11 | AUCA

Member	Student ID	Share of work
Christian Rwibutso Hakizinka	101373	33.3 %
Ariane Iradukunda	101390	33.3 %
Suavis Iradukunda	101391	33.3 %

GitHub repository: https://github.com/cikristian/Big_Data_Analytics_Project2_Group11 Submission date: 28 October 2026

Summary

We captured the fixed teaching dataset (**20,993,500 records**, including **20,003,300 transfers**) and loaded it into PostgreSQL 18.6 and Neo4j Community 2026.09.0. Each investigation question Q1 to Q7 is a parameterized SQL/Cypher pair. All 61 correctness cases were also recomputed independently from the captured CSVs with pandas, and the PostgreSQL answers matched every one.

The six investigation sets form three look-alike pairs. Their device sharing (Q3), network reach (Q4) and shortest routes (Q7) are almost identical. They differ in money movement: review accounts A, B and C send **200,000 to 1,050,000 RWF** onward in **40-second steps** (Q5, Q6), while controls D, E and F move **30,000 to 45,000 RWF** with **two days** between steps.

We recommend PostgreSQL as the system of record for daily reporting, and Neo4j as a periodically refreshed copy for multi-hop investigation queries (section 8.2).

1 Workspace and capture

1.1 Workspace

Item	Value
Operating system	Windows 11
CPU / RAM	8 logical processors; 15.84 GiB RAM (reported by neo4j-admin)
Python	3.13 in a project virtual environment; psycopg 3, the official neo4j Python driver, numpy, pandas and matplotlib
PostgreSQL	18.6, local Windows service, database `mobile_money`, timezone **Africa/Kigali**; default memory settings
Neo4j	Community Edition 2026.09.0 (server zip in C:\neo4j); heap 2 GB initial = max, page cache 3 GB
Java	Azul Zulu 21.0.11 (the Java 21 runtime bundled with Neo4j Desktop; the system Java 17 cannot run Neo4j 2026)
Data folder	C:\mm_work\data, outside OneDrive so sync does not interfere with 1.6 GB of CSVs

1.2 Capture

`capture.py` calls each generator once and writes each yielded dictionary straight to CSV with `csv.DictWriter`, so it never holds more than one row in memory. For every stream it counts rows consumed and written and compares both with `dataset_info()`. For transfers it also accumulates the audit totals that are used later to check both databases.

Stream	Expected	Written	Match
customers	228,000	228,000	yes
accounts	248,000	248,000	yes
devices	228,600	228,600	yes
account_devices	285,600	285,600	yes
transfers	20,003,300	20,003,300	yes
Total	**20,993,500**	**20,993,500**	yes

Transfer audit	Value
count	20,003,300
earliest occurred_at	1788235200 = 1 Sep 2026 06:00:00 CAT
latest occurred_at	1790798399 = 30 Sep 2026 21:59:59 CAT
sum amount_rwf	231,482,327,870 RWF

Check that caught an error. Our first PostgreSQL load began while `transfers.csv` was still being written, and it copied only 19,512,962 rows. The row-count check exposed this. We confirmed the finished file had 20,003,301 lines (rows plus header) and rebuilt the tables. Every figure below comes from the complete rebuild.

2 PostgreSQL

2.1 Schema

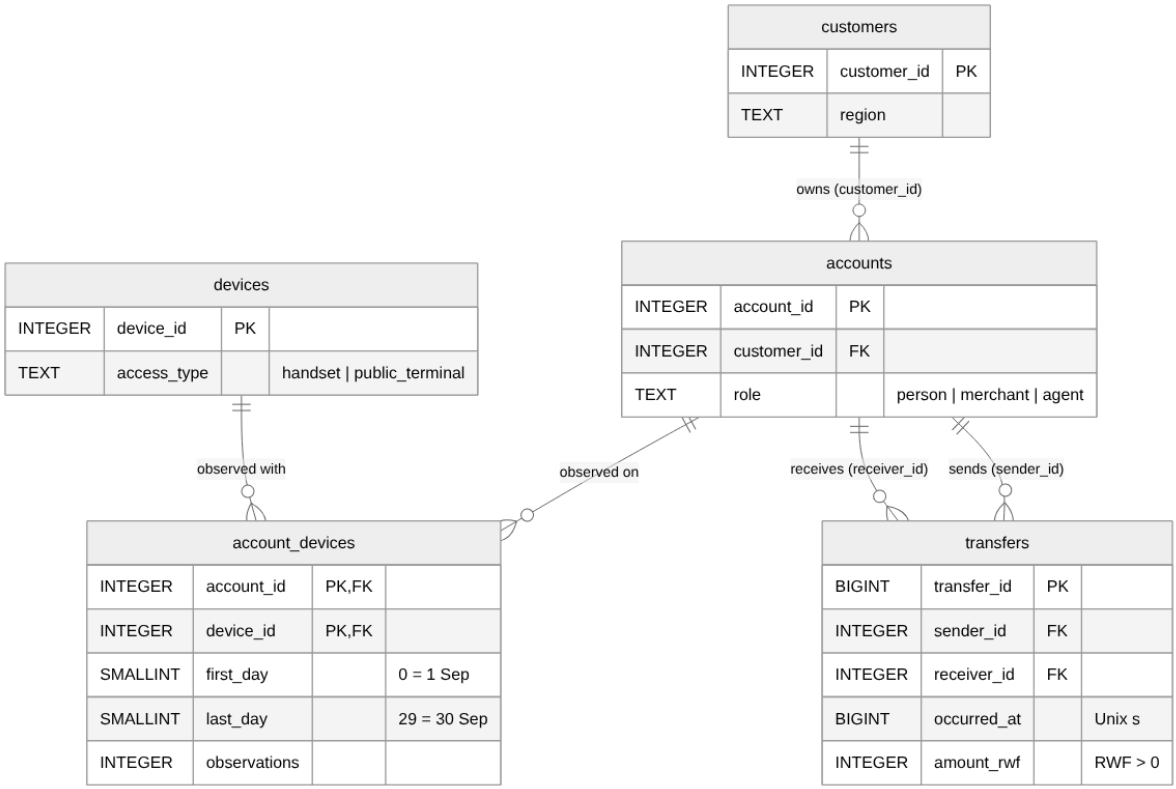
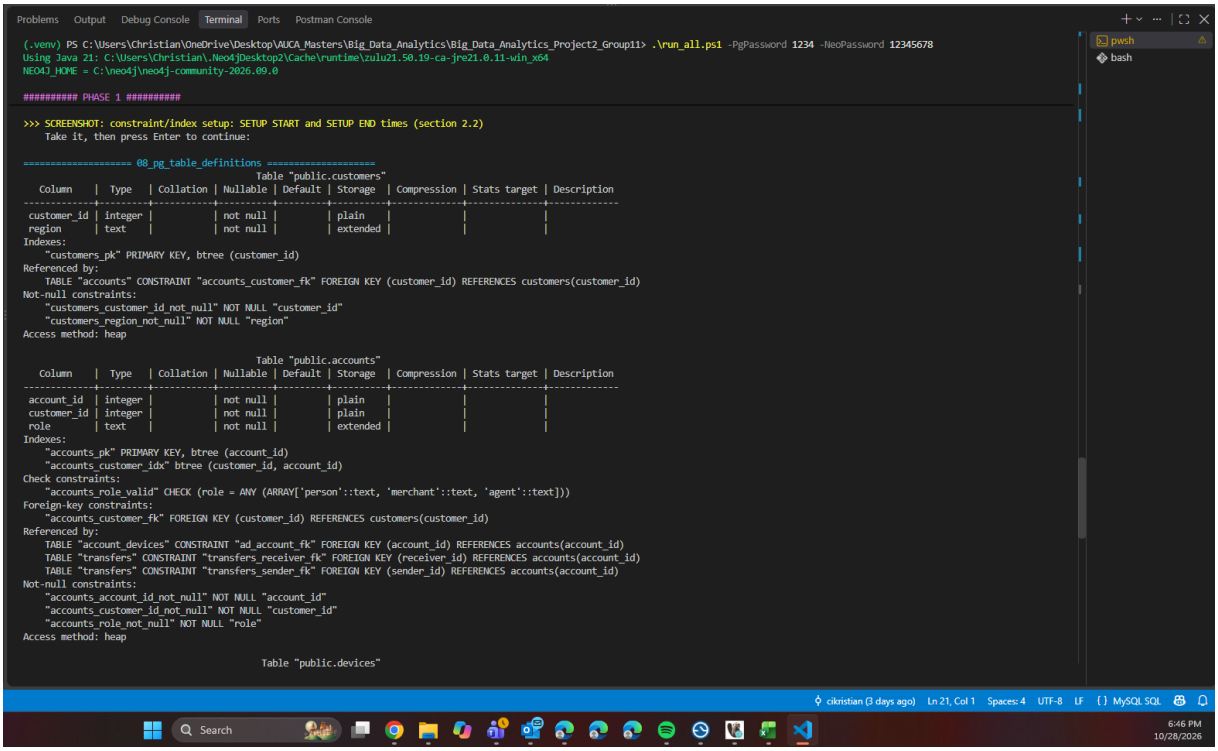


Figure 1. Relational schema: five tables, primary keys, foreign keys and units

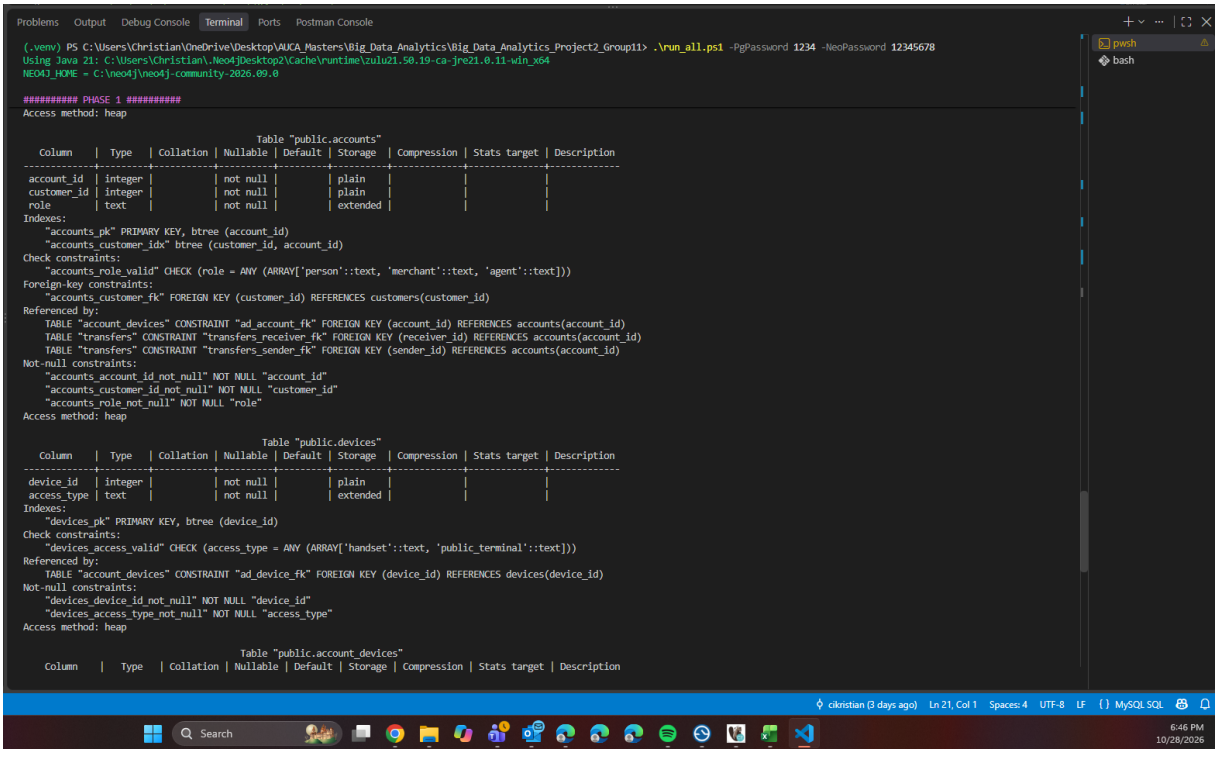
Field	Meaning and unit
customers.customer_id / region	customer identifier; one of five Rwandan regions
accounts.account_id / customer_id / role	wallet; its owning customer; person, merchant or agent
devices.device_id / access_type	observed device; handset or public_terminal
account_devices.first_day / last_day	inclusive observation days, 0 = 1 Sep 2026 CAT … 29 = 30 Sep
account_devices.observations	number of observations of the account on the device (> 0)
transfers.occurred_at	Unix epoch seconds (an instant; CAT = UTC+2)
transfers.amount_rwf	whole Rwandan francs, > 0

Tables are created with NOT NULL columns only, then loaded, and then given their keys, foreign keys and CHECK rules (`0 <= first_day <= last_day <= 29`, `observations > 0`, `amount_rwf > 0`, valid role and access type). This keeps the load fast and times it separately from the setup.

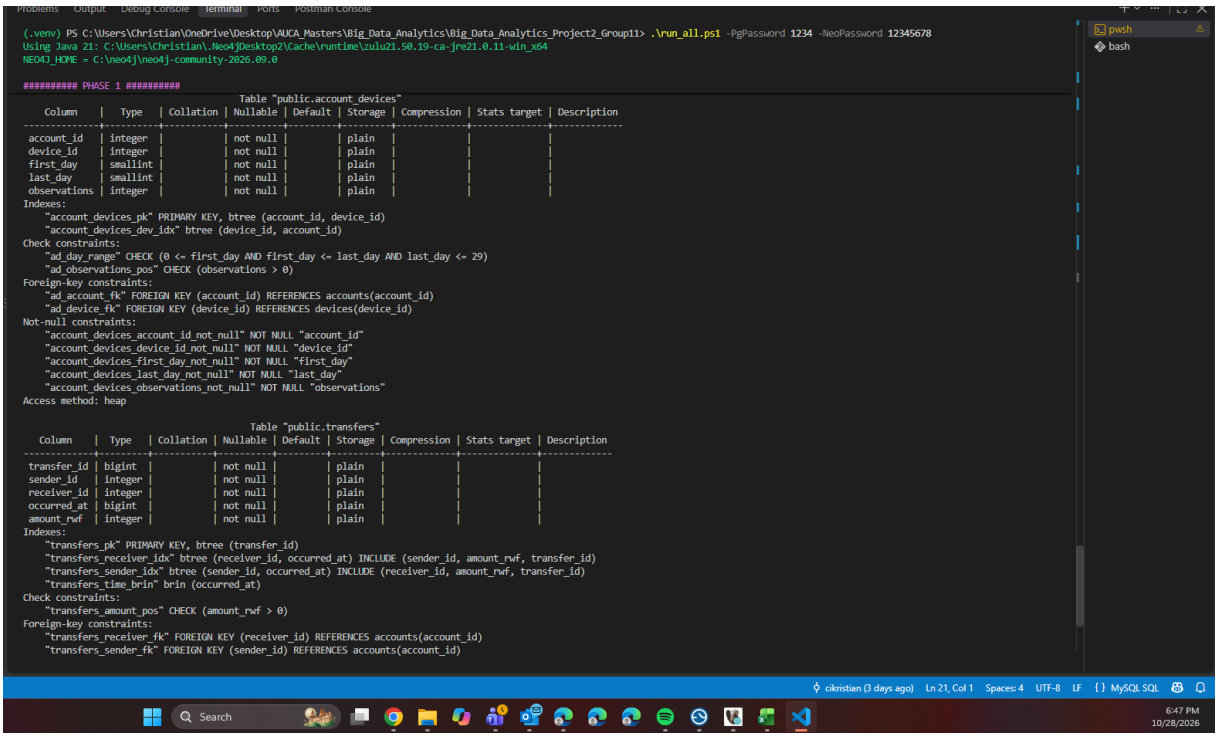
The secondary indexes follow the questions. `(sender_id, occurred_at)` and `(receiver_id, occurred_at)` include the remaining transfer columns, so the forward and backward transfer hops in Q1 and Q5 to Q7 are answered from the index alone. `accounts(customer_id)` and `account_devices(device_id)` serve the reverse lookups in Q3 and Q4. A BRIN index on `occurred_at` covers time ranges.



Screenshot 1. \d+ definitions: customers and accounts



Screenshot 2. \d+ definitions: devices



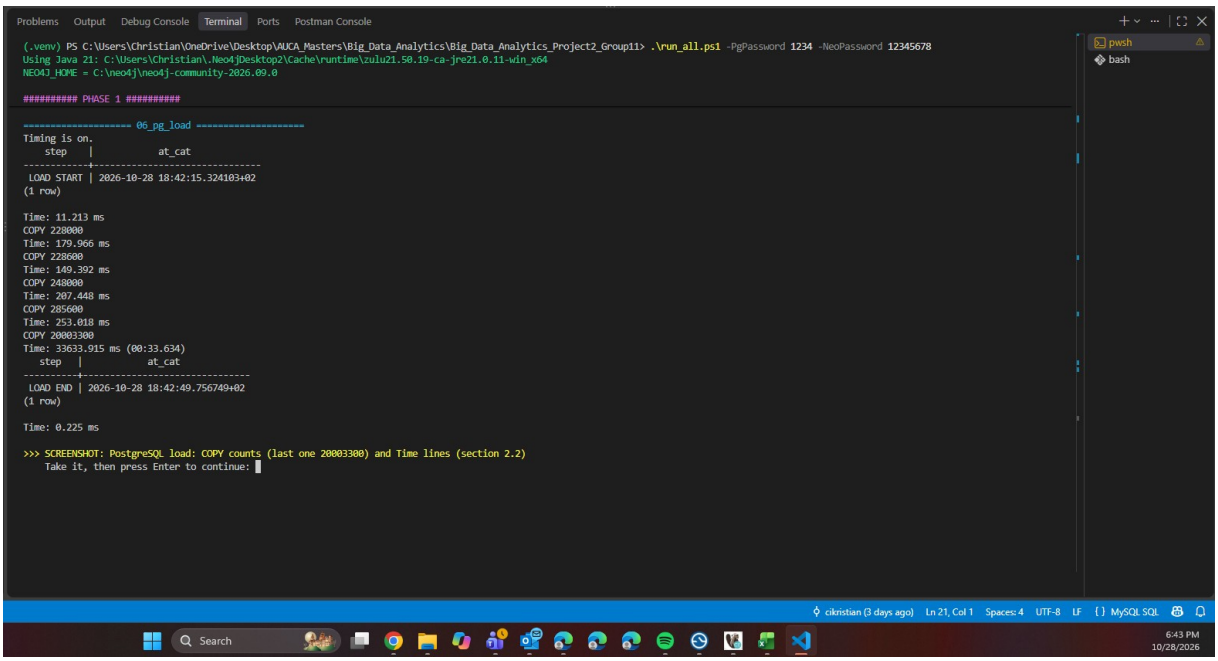
Screenshot 3. \d+ definitions: account_devices and transfers

2.2 Load and check

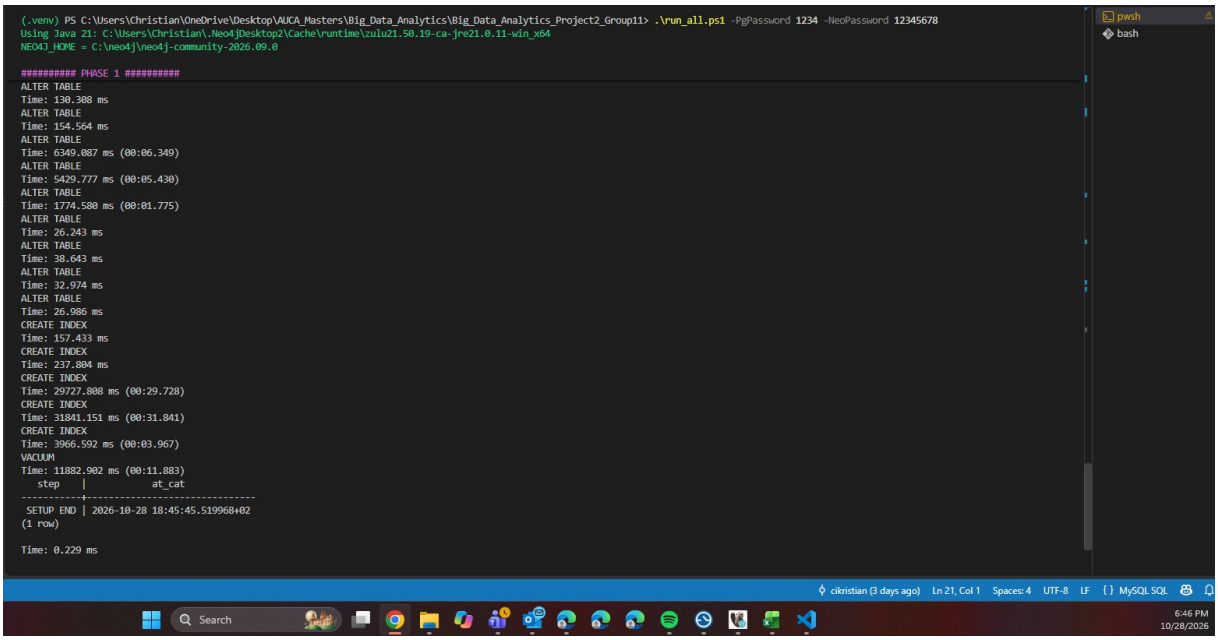
Step (psql \timing)	Seconds
\copy customers / devices / accounts / account_devices	0.18 / 0.15 / 0.21 / 0.25
\copy transfers (20,003,300 rows)	33.63
Load total (LOAD START 18:42:15.32 to LOAD END 18:42:49.76)	**34.43**
Largest setup steps: transfers_sender_idx / transfers_receiver_idx	29.73 / 31.84
VACUUM ANALYZE / transfer foreign keys / BRIN index	11.88 / 6.35 + 5.43 / 3.97

Table	Data GB	Index GB
transfers	1.2057	2.4335
account_devices	0.0127	0.0129
accounts	0.0111	0.0112
devices	0.0102	0.0052
customers	0.0101	0.0051
Database total	**3.7262**	

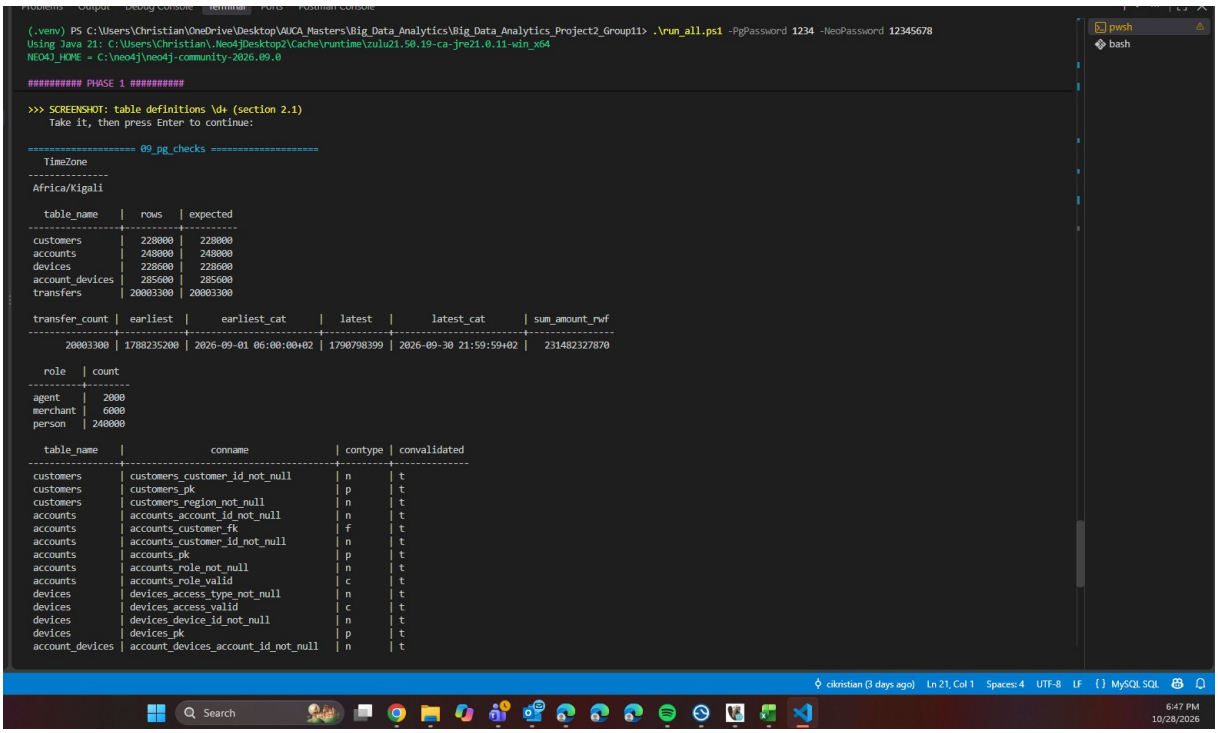
All five row counts match the capture. The transfer audit matches exactly (20,003,300 rows; earliest 1788235200; latest 1790798399; sum 231,482,327,870 RWF). The roles split into 240,000 person, 6,000 merchant and 2,000 agent accounts, and every constraint shows `convalidated = t`. The two covering transfer indexes are twice the size of the table they index. That is the storage cost of answering traversal hops without reading the table.



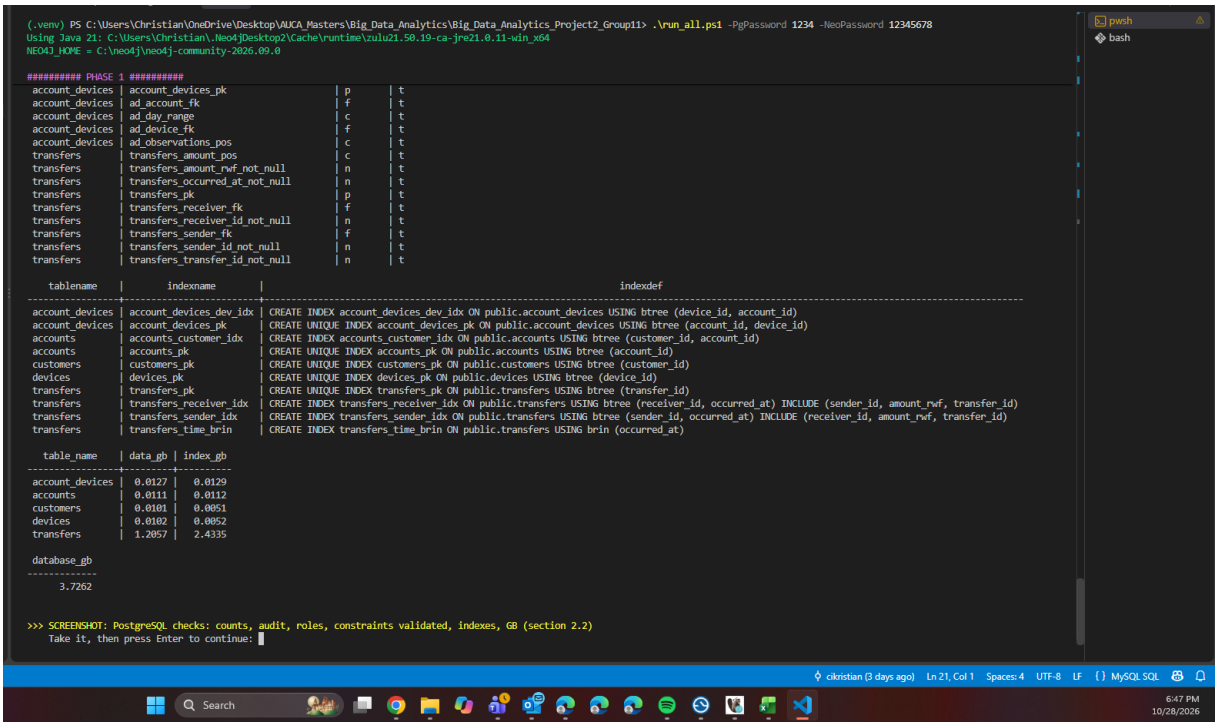
Screenshot 4. Bulk load: COPY counts and timings



Screenshot 5. Constraint and index setup timings



Screenshot 6. Checks: counts, transfer audit, roles, validated constraints



Screenshot 7. Checks: indexes and sizes in GB

3 Neo4j

3.1 Graph mapping

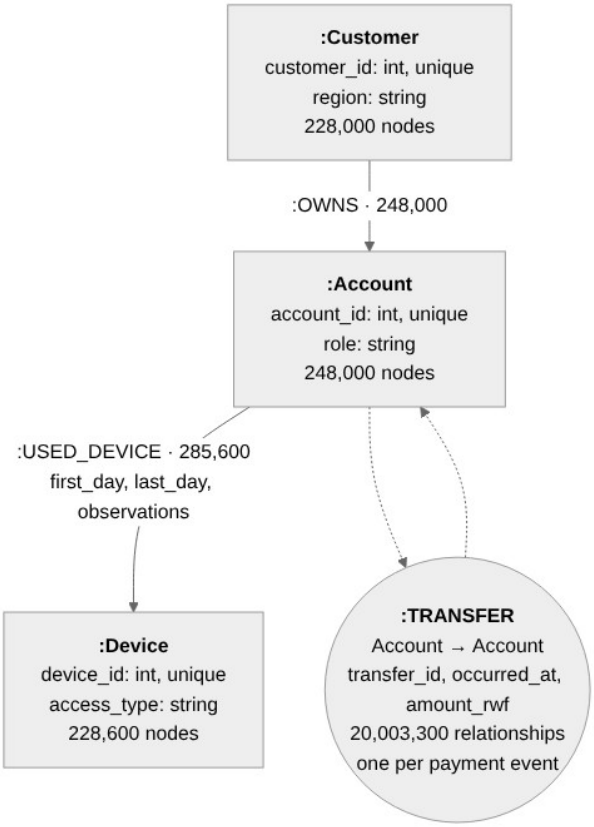


Figure 2. Property-graph model

Source	Graph element
customers row	:Customer {customer_id, region}
accounts row	:Account {account_id, role}; accounts.customer_id becomes (:Customer)-[:OWNS]->(:Account)
devices row	:Device {device_id, access_type}
account_devices row	(:Account)-[:USED_DEVICE {first_day, last_day, observations}]->(:Device)
transfers row	(sender)-[:TRANSFER {transfer_id, occurred_at, amount_rwf}]->(receiver), one relationship per event

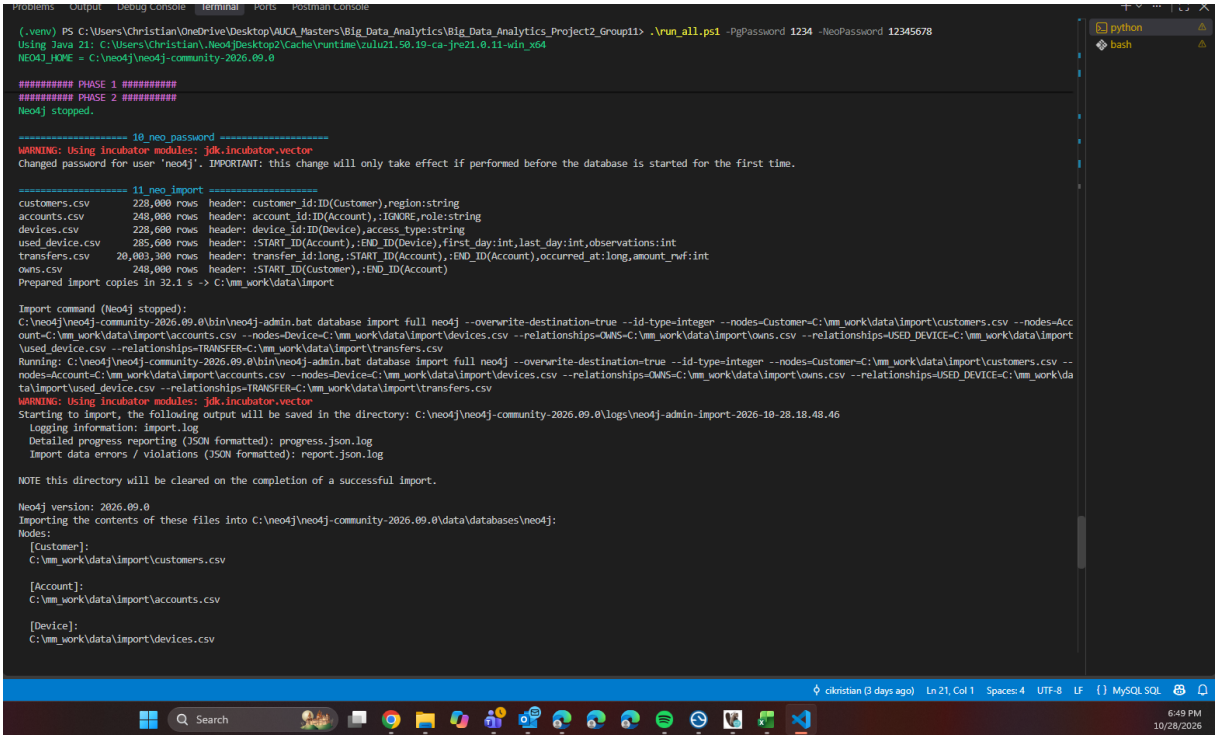
3.2 Import

`prepare_import.py` writes typed copies to `data\import` and leaves the source CSVs unchanged. It uses ID spaces `ID(Customer)`, `ID(Account)` and `ID(Device)`, `:IGNORE` for the owner column of accounts, `:int` and `:long` numeric types, and a derived `owns.csv` ordered by customer_id and then account_id. It then runs `neo4j-admin database import full` with Neo4j stopped. After start-up we created unique constraints on the three business IDs and a range index on TRANSFER`.occurred_at`, and waited until all were online.

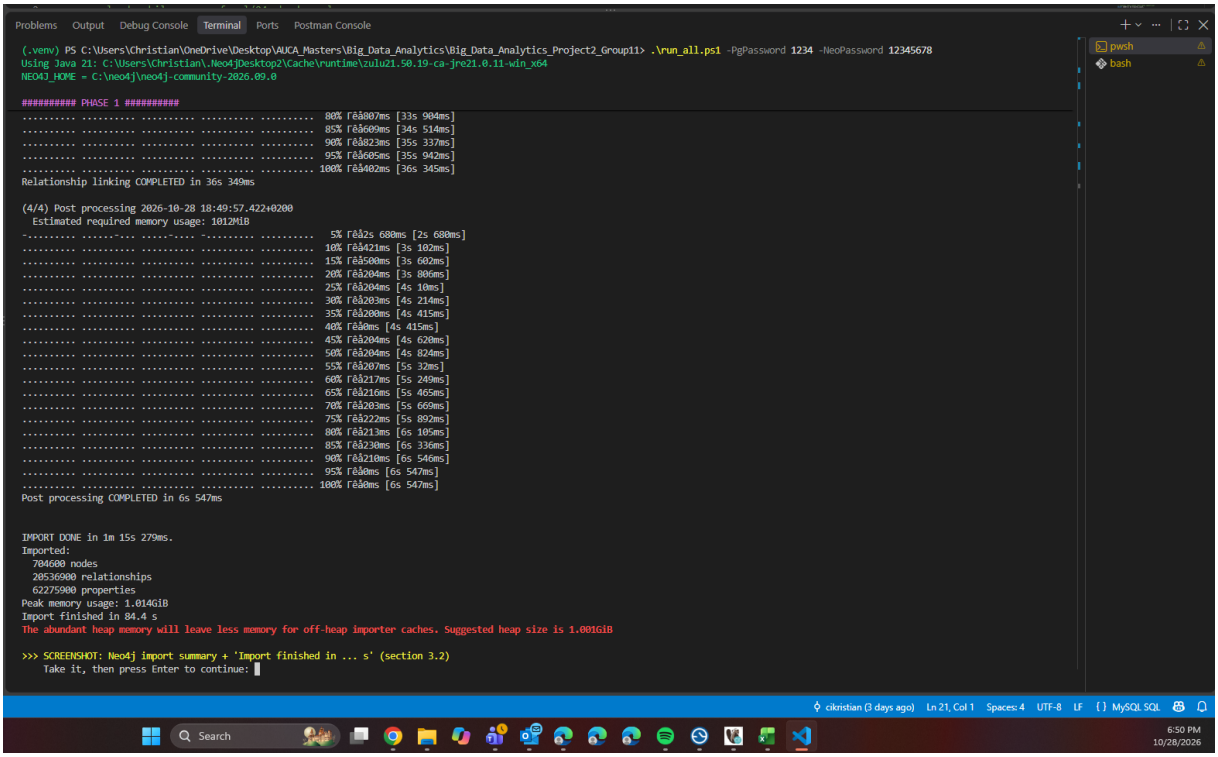
Measure	Value
Typed import copies prepared	32.1 s
neo4j-admin import (IMPORT DONE 1 min 15.3 s; process total)	**84.4 s**
Imported	704,600 nodes; 20,536,900 relationships; 62,275,900 properties

Measure	Value
Peak importer memory	1.014 GiB

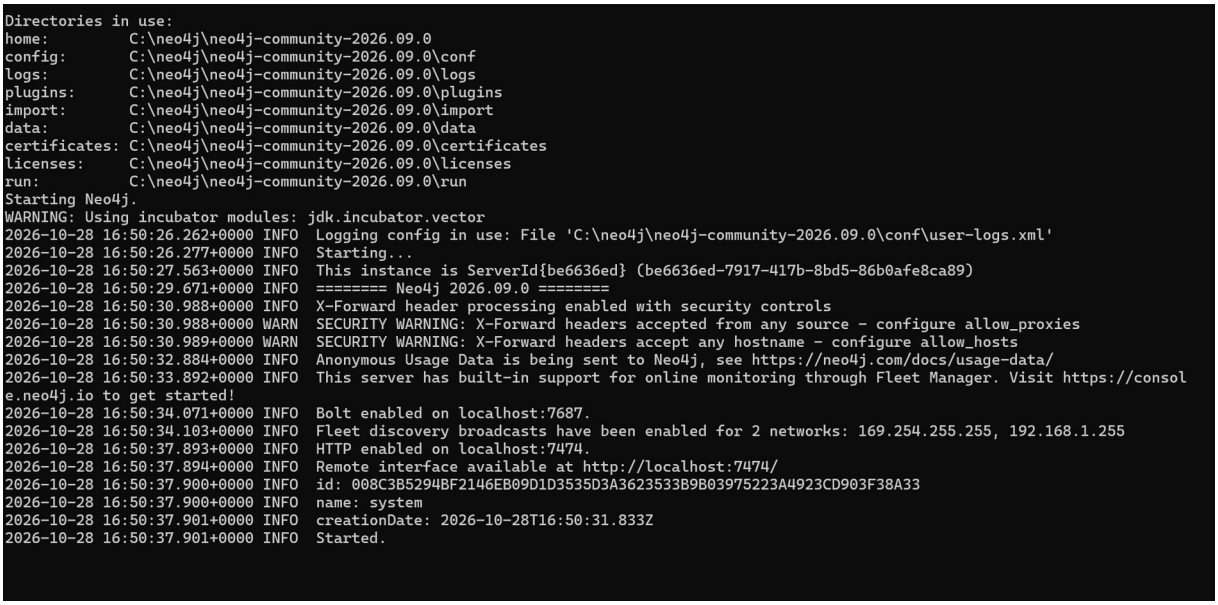
The importer reported Cypher type normalisation from `int` to `long` for four properties. The values are unchanged; only their stored width differs.



Screenshot 8. Neo4j password set, typed import copies and import command



Screenshot 9. Import completed: 704,600 nodes and 20,536,900 relationships



Screenshot 10. Neo4j Community 2026.09.0 started

3.3 One account in both models

In PostgreSQL, `accounts.customer_id` is a foreign-key value, and the database finds the owner by searching the `customers_pk` index during a join. In Neo4j the same fact is stored as an OWNS relationship, so the query follows a stored pointer from the account to its owner.

We keep **one TRANSFER relationship per payment**. If repeated payments between two accounts were collapsed into one weighted edge, each event's ID, time and amount would be lost. Q1 and Q2 totals could no longer be filtered by hour or day. The Q5 rules (600-second gap, amount within 90 to 105% of the previous) and the Q6 strictly increasing times could not be evaluated, and Q7 path checks could not name the transfers used.

4 Routine questions

Each set's review hour runs from 09:45:00 (inclusive) to 10:45:00 (exclusive) CAT on its September day. The bounds are computed in Python as epoch seconds (`config.hour_bounds`) and passed to both engines as parameters. Appendix A lists every query pair.

4.1 Q1 Outgoing transfers in the review hour

Set	Account	Day (Sep)	Transfers	Total RWF
A	998	6	1	200,000
B	147228	13	1	1,050,000
C	206978	8	1	300,000
D	237998	6	1	45,000
E	144228	13	1	40,000
F	203978	8	1	30,000

Outgoing value is not a balance. The data contain no opening balance and no incoming transfers in this calculation, and they record no fees, cash-in or cash-out, or reversals. The transfers are wallet-to-wallet events, not a complete ledger.

4.2 Q2 Daily activity

Q2 returns 30 days. Daily counts range from 666,666 to 666,997, and daily value from 7.66 to 7.86 billion RWF. The days sum to **20,003,300 transfers** and **231,482,327,870 RWF**, equal to the capture audit.

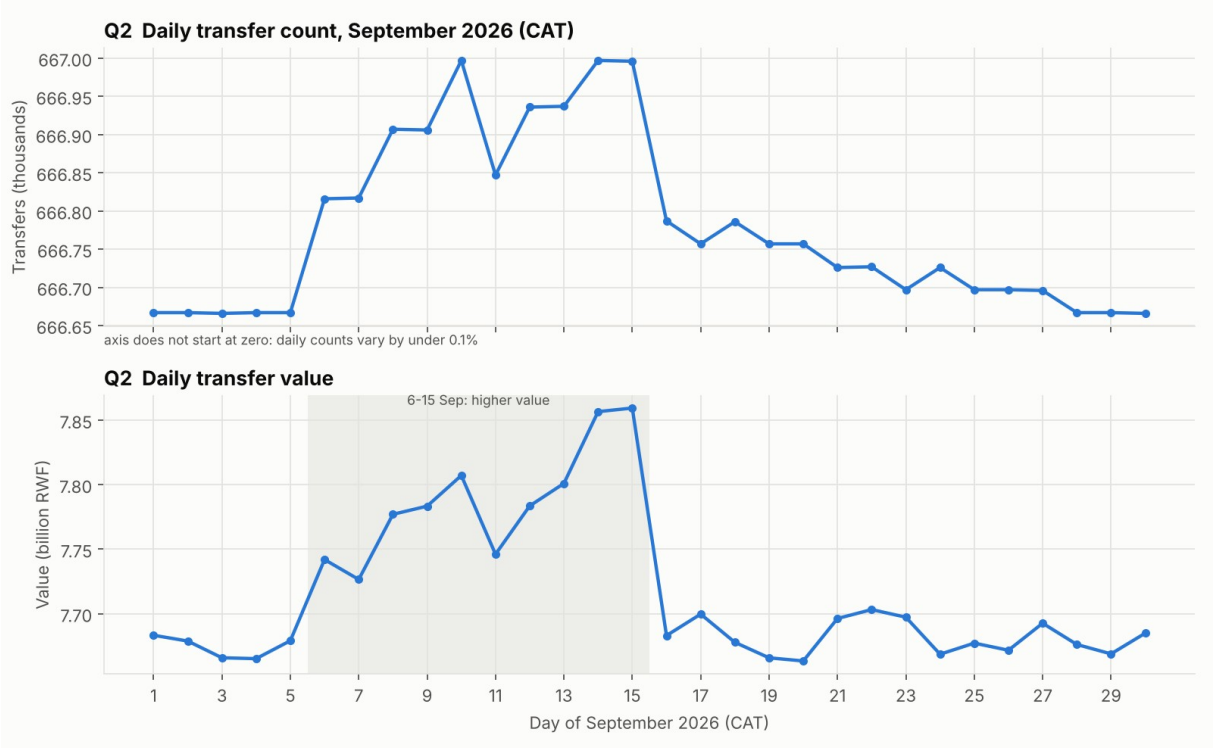


Figure 3. Q2 daily transfer count (top) and value (bottom), September 2026 CAT

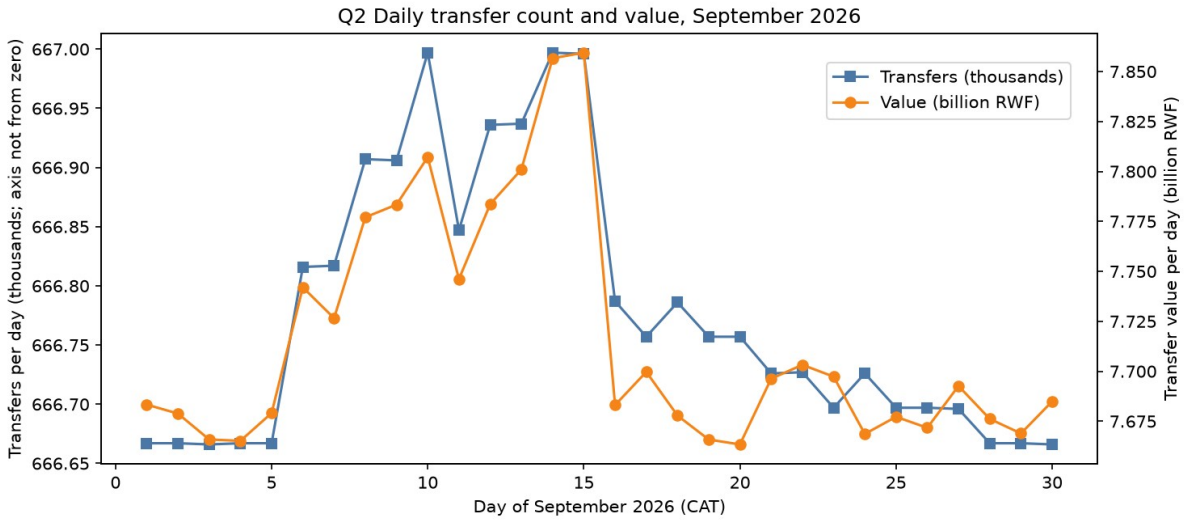


Figure 3b. The same Q2 result from our PostgreSQL run (plots.py on our machine); count on the left axis, value on the right axis, neither starting at zero

Figure 3b, drawn from our own PostgreSQL database, shows the same 30 daily points as Figure 3: count peaks near 667,000 on 10, 14 and 15 September, value peaks at 7.86 billion RWF on 14 and 15 September, and both fall back on the 16th. Because its two y-axes use unrelated scales, read the lines separately rather than comparing their heights.

Pattern. On 6 to 15 September the daily value rises by up to about 2% while the count rises by fewer than 350 transfers (under 0.05%). A small number of extra large transfers therefore moves the total: the investigation scenarios run on those days with amounts of 200,000 to 1,050,000 RWF. Daily value is **not revenue**. Revenue comes from fees and commissions, which these records do not contain, and moving the same money twice counts twice in value.

4.3 Q3 Shared devices

Set	Rows	Devices shared (accounts seen on them with overlapping days)
A	4	handset 998 with 220998 (the same customer's second wallet); handset 228001 with 8917, 16836, 24755
B	9	household handset 147226 with 147226, 147227, 147229, 147230; handset 228018 with 155147, 163066, 170985, 178904, 186823
C	9	household handset 206976 with four household accounts (206976 from day 11); handset 228043 with 6573, 214897, 222816, 230735, 238654
D	4	handset 17998 with 17998 (237998 is customer 17998's second wallet); handset 228301 with 5917, 13836, 21755
E	9	household handset 144226 with four household accounts; handset 228318 with five accounts
F	9	household handset 203976 with four household accounts (203976 from day 11); handset 228343 with five accounts

Legitimate sharing. A family of five shares one phone, which is the household handset in B, C, E and F. A customer may also run a second wallet on their own phone, as in A and D. Overlapping intervals only mean both accounts were observed at some point within the common days. They do **not** show the two accounts were used at the same moment, or that the same person used both.

5 Connected accounts

5.1 Q4 Accounts reached through ownership and devices

Set	Depth 4	Depth 6	Depth 8	Depth 10
A	10	43	175	187
B	46	178	188	228
C	146	483	832	1,433
D	10	44	180	192
E	47	183	193	230
F	147	488	837	1,426

Distinct other accounts reached, summed over the five regions (per-region rows are in the screenshots)

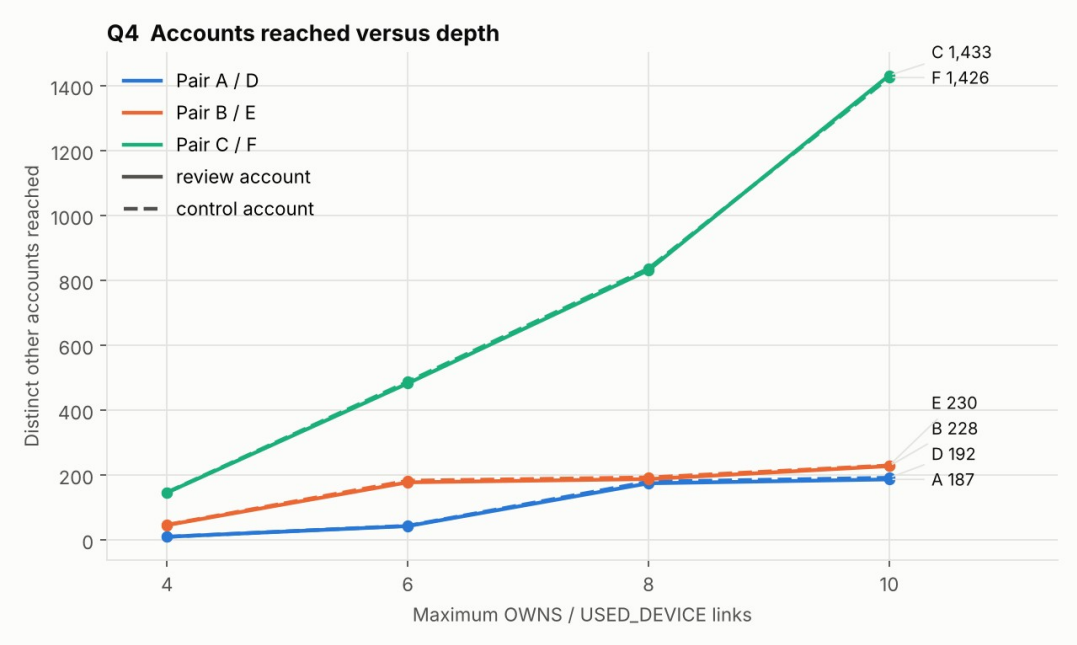


Figure 4. Q4 reach versus depth; review and control accounts of each pair almost coincide

Weak associations. Each extra pair of links adds a customer or a device that someone else also used. Sets C and F grow fastest because household members' devices fan out widely. For C, household account 206980 used **public terminal 220020** (60 accounts), and 206976's **replaced handset 223689** was later used by 34 accounts. After 8 to 10 links the set contains strangers whose only connection is a shared kiosk or a second-hand phone. Reach measures **exposure, not relationship**, and the near-identical curves of each pair show it cannot separate review from control accounts.

5.2 Q5 Rapid three-transfer chains

Set	Matches	Chain (t1, t2, t3 → final receiver)
A	1	20000001, 20000002, 20000003 → 24755
B	1	20000092, 20000093, 20000094 → 170985
C	1	20000229, 20000230, 20000231 → 230735
D	0	none
E	0	none
F	0	none

Transfer	From → to	Time (13 Sep 2026 CAT)	Amount RWF	% of previous
20000092	147228 → 155147	10:00:17	1,050,000	
20000093	155147 → 163066	10:00:57	1,029,000	98%
20000094	163066 → 170985	10:01:37	1,008,420	98%

Example chain for set B: four distinct accounts, 40 s between steps, amounts within the 90 to 105% rule

The chain does not prove the **same funds** moved onward. Wallet money is interchangeable: 155147 may have paid 163066 from an existing balance, and the records show no balances. The pattern is consistent with pass-through, but also with unrelated payments that happen to be close in time and size.

5.3 Q6 Rapid cycles

Set	Matches	Cycle transfer IDs	Length
A	1	20000001, 20000002, 20000003, 20000004	4
B	0	none (the chain does not return)	
C	1	20000229, 20000230, 20000231, 20000232, 20000233, 20000234	6
D	0	none (the loop exists but takes 6 days)	
E	0	none	
F	0	none (the loop exists but takes 10 days)	

Set C's cycle leaves 206978 at 10:00:42 with 300,000 RWF and returns at 10:04:02 with 271,176 RWF. That is six hops in 200 seconds, each 98% of the previous one (Figure 5 in section 8).

Further evidence needed: KYC links between the six holders; whether 206978 cashed out after the return; balances before and after; which device initiated each payment (observations do not say this); and customer complaints or reversals.

5.4 Q7 Shortest person-only route

Set	Source → destination	Bound 4	Bound 6
A	998 → 24755	3	3
B	147228 → 186823	no route	5
C	206978 → 6573	no route	5
D	237998 → 21755	3	3
E	144228 → 183823	no route	5
F	203978 → 3573	no route	5

No route within 4 does not mean no connection. B, C, E and F connect at length 5. The route is also **structural**: it may join transfers made on different days, ignores amounts, and could use a different but equally short path. So it is not evidence that money flowed from the source to the destination.

6 Correctness and performance

6.1 Matching results

`compare.py` runs all 61 cases on both engines through `run_pg` and `run_neo`, normalises the values (Decimal to int, dates to ISO strings, lists to tuples), sorts the rows and compares them exactly. It also compares each case with `reference.py`, an independent pandas/NumPy recomputation from the captured CSVs. Q7 lengths are compared, and every Neo4j supporting path is then checked against PostgreSQL rows: endpoints, length, direction of each transfer, and person role of every account.

`verify_csv.py` re-reads the raw CSVs without any database. Device 228001 was observed with 998 on days 0 to 29 (16 observations) and with 24755 on days 0 to 29 (7 observations), so they overlap on days 0 to 29. Chain A (20000001 to 20000003) and cycle C (20000229 to 20000234) satisfy every rule: linked accounts, increasing times, gaps of 600 s or less, amounts within 90 to 105%, distinct accounts or a single return, and a span of 1,800 s or less.

6.2 Fixed-resource timings

We timed 21 cases (B and E for Q1, Q3, Q4 at four depths, Q5, Q6 and Q7 at two bounds, plus the Q2 month) with **only one database service running**. Each case had three untimed warm-ups (180 s limit) and five measured runs (60 s limit). Timing runs from submission until the last row is fetched; answers were checked against the reference outside the timer, and failed or timed-out runs are not counted. Raw runs are in `results\bench_raw_*.csv`.

Processing difference (Q7 B bound 6). PostgreSQL's recursive CTE expands every account-depth state reachable in six hops and joins each new transfer to `accounts` to test the role, before it filters for the destination. Neo4j's `shortestPath` runs a breadth-first search from both ends that stops when the two frontiers meet, so it touches only a small part of the graph.

6.3 Improving one slow query

Wasted work in the baseline Q7 SQL. In a preliminary verification run of the same data (PostgreSQL 16 on a separate Linux machine), the recursive union produced about 428,000 account-depth rows and probed `accounts_pk` about 14.6 million times (about 60 million buffer hits). It kept expanding after the destination was already reachable, and it revisited accounts at every depth.

Revision (`Q7_SQL_BIDIR`). The revised query uses only distinct neighbour accounts. It grows a forward set of radius $\lceil b/2 \rceil$ from the source and a backward set of radius $\lfloor b/2 \rfloor$ into the destination, both restricted to person accounts, and returns the smallest forward-plus-backward depth over accounts in both sets. It is still correct: any qualifying route of length $L \le b$ has an account at position $\max(0, L - \lfloor b/2 \rfloor)$ that lies in both sets, and no shorter total can exist. All 12 Q7 cases still match.

In that run the bound-6 query fell from about 15 to 20 s to about 30 ms with identical answers, because each account is expanded once and the two searches stop halfway. The revised version is the one used in section 7 and in the final comparison.

7 Concurrency

7.1 Load test

For each engine (the other stopped) and each workload (Q4 depth 10 and Q7 length-only bound 6, cycling sets A, B, C), we ran 1, 4, 8 and 16 clients three times. Each client is a separate process with its own persistent connection. It does one untimed A-B-C warm-up, waits at a shared barrier, then starts cycles until 15 s after release, finishing any cycle already started. Throughput is correct requests divided by the time from release to the last cycle finishing. Median and p95 latency use completed requests. Each cell below is the median of the three repetitions.

7.2 Scaling

On 8 logical processors, throughput should rise until the clients use all cores and then flatten, while p95 keeps rising because requests queue for CPU. **Throughput** is the capacity of the whole system; **p95** is the wait an individual analyst experiences. These claims apply only to this machine, these memory settings and these two queries.

8 Decisions

8.1 Investigation priority

Pair	Q3 devices	Q4 depth 10	Q5 / Q6	Q7 (b4 / b6)	Review first
A / D	4 / 4 rows; own second wallet + one shared handset	187 / 192	A: chain and 4-step cycle in 2 min, 200,000 → 188,238 RWF. D: none; same loop takes 6 days at 45,000 RWF	3/3 and 3/3	**A**
B / E	9 / 9 rows; household + one shared handset	228 / 230	B: chain 1,050,000 → 1,008,420 RWF in 80 s; no cycle. E: none; 40,000 RWF every 2 days	none/5 and none/5	**B**
C / F	9 / 9 rows; household + one shared handset	1,433 / 1,426	C: chain and 6-step cycle in 200 s, 300,000 → 271,176 RWF. F: none; loop of 30,000 RWF over 10 days	none/5 and none/5	**C**

Within every pair the structural evidence (Q3, Q4, Q7) is the same, so it cannot set priority. The timing and amount evidence (Q5, Q6) does. Across the pairs we would review **C** first, because it is a closed cycle that returns most of a large amount within 200 seconds; then **B**, the largest single amount moved onward; then **A**.

Pair C/F in detail. Both 206978 and 203978 belong to a five-account household handset (206976 / 203976) and share a second handset (228043 / 228343) with the five accounts on their route. Both reach about 1,430 accounts within ten links and reach their destination (6573 / 3573) in five person-to-person transfers. The difference is behaviour. On 8 September, 206978 sent 300,000 RWF at 10:00:42, and the money passed 214897, 222816, 230735, 238654 and 6573 before 271,176 RWF returned at 10:04:02, each step 40 s after the previous one. F's accounts form the same loop, but at a flat 30,000 RWF with two days between payments (8 to 18 September).

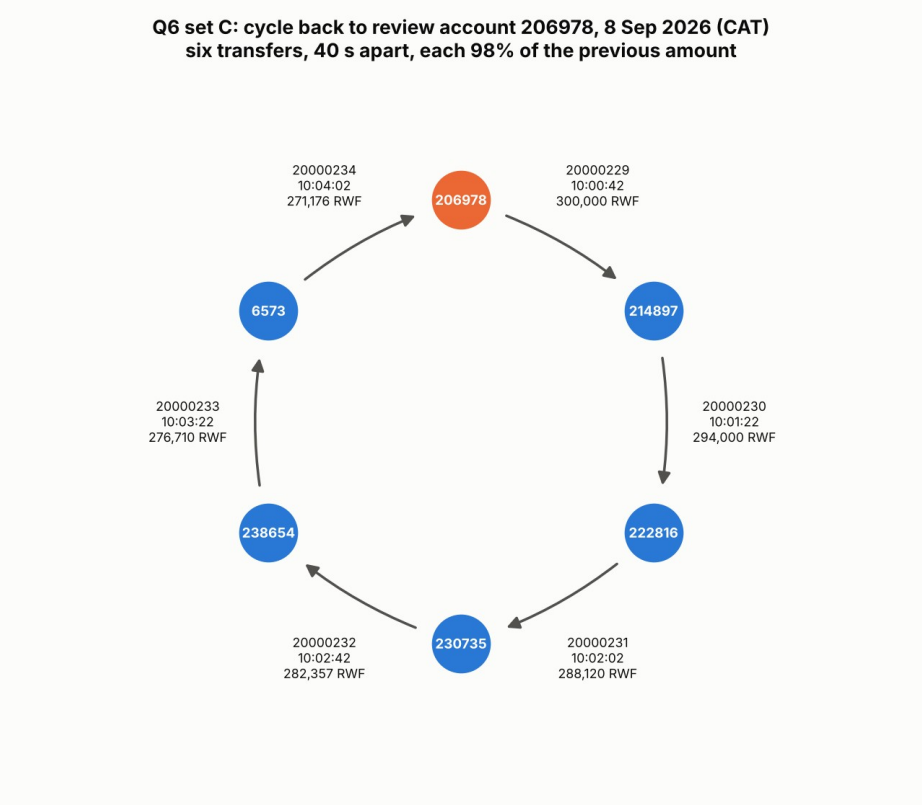


Figure 5. Transfer cycle for set C (Q6)

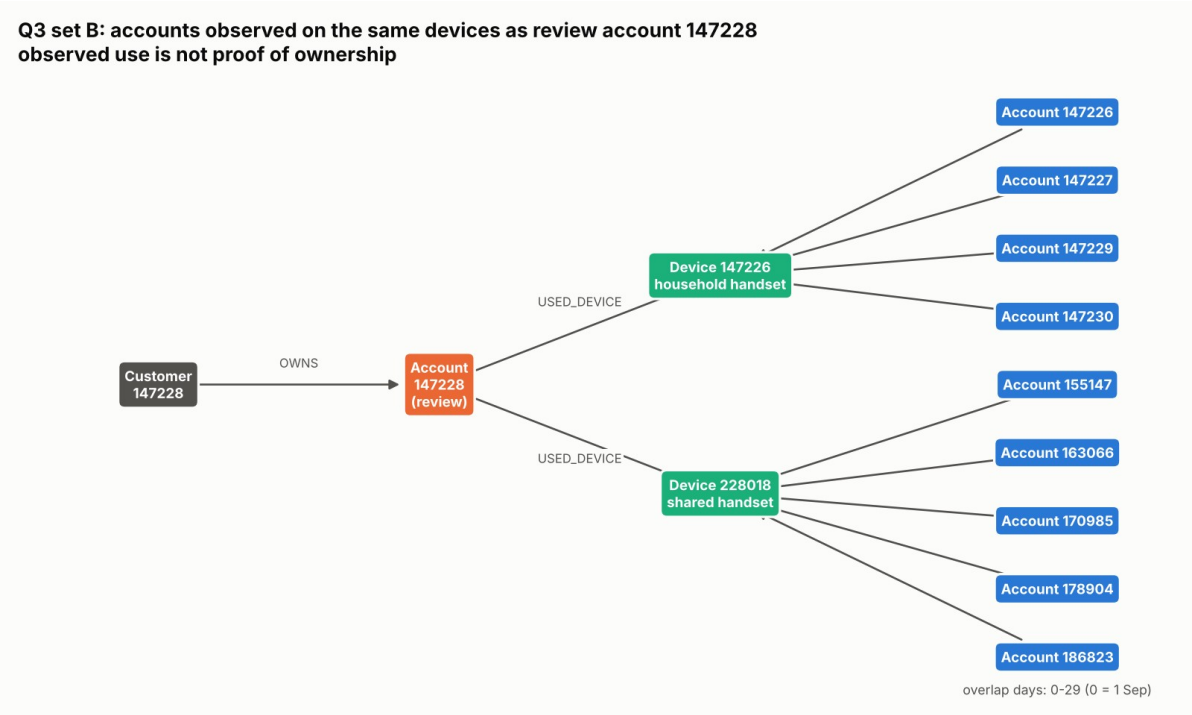


Figure 6. Ownership and device links for set B (Q3)

Ordinary explanation. The six accounts in cycle C could be a rotating savings group (ikimina). Members pass a contribution around quickly while meeting together, which also explains why they were all seen on one shared handset. The 2% fall per step could be a fee or a share kept at each step.

Missing evidence: KYC and relationship information on the holders, balances and cash-out after the cycle, the device that initiated each payment, and whether the pattern recurs on other days. A recurring weekly pattern among known group members would lower our concern. An immediate cash-out by a recently opened account would raise it. These records alone **do not show fraud**; they show which account to review first.

8.2 Recommendation (at most 500 words)

Reporting (Q1 to Q3) → PostgreSQL. Q2 groups all 20 million transfers by day, a set-based scan that a relational engine with a time index handles well, while Cypher must visit every TRANSFER relationship. Q1 and Q3 are short index lookups that either engine answers quickly. Loading took 34.4 s in PostgreSQL against 32.1 s of preparation and an 84.4 s import in Neo4j, and PostgreSQL's keys, foreign keys and CHECK rules keep the reporting data clean.

Traversal (Q4 to Q7) → Neo4j. Variable-length patterns such as `[:OWNS|USED_DEVICE*1..10]` and `shortestPath` express multi-hop questions directly and follow stored relationships instead of repeating joins. Part of any speed gap is the **algorithm, not the model**: rewriting Q7 as a two-ended SQL search cut it from about 15 to 20 s to about 30 ms in our verification run. We therefore credit Neo4j mainly with shorter, safer traversal queries.

Concurrency. Each analyst uses a separate connection on both engines. On a machine with 8 logical processors, heavy multi-hop traversals compete with reporting queries for CPU, which is a further reason to run them on the graph copy rather than on the reporting database.

Evidence against our choice. The improved SQL shows PostgreSQL can answer the hardest traversal efficiently when the query is written well, and short-hop questions (Q5, Q6) are fast on both engines. Running Neo4j as well means a second store to load, secure and keep consistent with PostgreSQL's 3.73 GB.

Freshness. Copy new transfers from PostgreSQL, which stays the system of record, into the graph at least every **15 minutes**. A stale graph misses recent chains and cycles, which are exactly the fast patterns investigators care about, and it can disagree with the daily reports. Investigators should see the graph's last-updated time next to every result.

Limits. These results come from one laptop with fixed memory settings, and from one synthetic month of data with planted scenarios. Real data volumes, skew, concurrent writes and memory tuning could change the ranking.

8.3 Individual judgement

Member	Student ID	Share of work
Christian Rwibutso Hakizinka	101373	33.3 %
Ariane Iradukunda	101390	33.3 %
Suavis Iradukunda	101391	33.3 %

External and LLM assistance. An AI assistant (Claude, Anthropic) helped draft the capture, import, query, benchmark and load-test programs, and an outline of this report. We checked the work in three ways: (1) an independent pandas recomputation of all 61 cases from the raw CSVs (`reference.py`), which matched every PostgreSQL answer; (2) raw-CSV checks of one overlap, one chain and one cycle (`verify_csv.py`); and (3) row counts and audit totals in PostgreSQL, and node and relationship counts in Neo4j, equal to the capture. Every member has read the code and can explain it.

9 Reproducible submission

The GitHub repository https://github.com/cikristian/Big_Data_Analytics_Project2_Group11 and the matching ZIP contain the unchanged generator (SHA-256 `2ea0e59d2b7b987d84a3d151bd9d1ad7b3052bfbc25abf1466c8e94467e9455a`, checked by `verify_generator.py`), the capture and import programs, SQL and Cypher definitions, query pairs, the comparison, benchmark and load-test programs, raw results, logs and charts. Secrets appear only as environment-variable placeholders, and the CSVs and stores are kept locally. `README.md` lists the commands in execution order; `run_all.ps1` runs them end to end.

Appendix A Query pairs

SQL uses psycopg `%(name)s` parameters and Cypher uses `$name`. The Cypher variable-length bounds <depth> and <bound> are substituted from a fixed whitelist of integers (4, 6, 8, 10 and 4, 6), because Cypher cannot take them as parameters.

Q1 SQL

```
SELECT COUNT(*) AS transfers, COALESCE(SUM(amount_rwf), 0) AS total_rwf
FROM transfers
WHERE sender_id = %(account)s
  AND occurred_at >= %(start)s AND occurred_at < %(end)s;
```

Q1 Cypher

```
MATCH (a:Account {account_id: $account})
OPTIONAL MATCH (a)-[t:TRANSFER]->()
  WHERE t.occurred_at >= $start AND t.occurred_at < $end
RETURN count(t) AS transfers, coalesce(sum(t.amount_rwf), 0) AS total_rwf;
```

Q2 SQL

```
SELECT (to_timestamp(occurred_at) AT TIME ZONE 'Africa/Kigali')::date AS day,
       COUNT(*) AS transfers,
       SUM(amount_rwf::bigint) AS total_rwf
FROM transfers
GROUP BY 1
ORDER BY 1;
```

Q2 Cypher

```
MATCH ()-[t:TRANSFER]->()
WITH date(datetime({epochSeconds: t.occurred_at, timezone: 'Africa/Kigali'})) AS day,
     t.amount_rwf AS amount
RETURN day, count(*) AS transfers, sum(amount) AS total_rwf
ORDER BY day;
```

Q3 SQL

```
SELECT o.account_id                       AS other_account,
       o.device_id,
       d.access_type,
       GREATEST(r.first_day, o.first_day)  AS overlap_start,
       LEAST(r.last_day, o.last_day)       AS overlap_end
FROM account_devices r
JOIN account_devices o ON o.device_id = r.device_id AND o.account_id <> r.account_id
JOIN devices d         ON d.device_id = r.device_id
WHERE r.account_id = %(account)s
  AND GREATEST(r.first_day, o.first_day) <= LEAST(r.last_day, o.last_day)
ORDER BY other_account, o.device_id;
```

Q3 Cypher

```
MATCH (a:Account {account_id: $account})-[r1:USED_DEVICE]->(d:Device)<-[r2:USED_DEVICE]-(b:Account)
WHERE b <> a
WITH b, d,
     CASE WHEN r1.first_day > r2.first_day THEN r1.first_day ELSE r2.first_day END AS s,
     CASE WHEN r1.last_day  < r2.last_day  THEN r1.last_day  ELSE r2.last_day  END AS e
WHERE s <= e
RETURN b.account_id AS other_account, d.device_id AS device_id, d.access_type AS access_type,
       s AS overlap_start, e AS overlap_end
ORDER BY other_account, device_id;
```

Q4 SQL

```
WITH RECURSIVE edges(sk, si, dk, di) AS (
    SELECT 'Customer'::text, customer_id, 'Account'::text,  account_id  FROM accounts
    UNION ALL
    SELECT 'Account'::text,  account_id,  'Customer'::text, customer_id FROM accounts
    UNION ALL
    SELECT 'Account'::text,  account_id,  'Device'::text,   device_id   FROM account_devices
    UNION ALL
    SELECT 'Device'::text,   device_id,   'Account'::text,  account_id  FROM account_devices
```

```
), walk(kind, id, depth) AS (
    SELECT 'Account'::text, %(account)s::integer, 0
    UNION
    SELECT e.dk, e.di, w.depth + 1
    FROM walk w
    JOIN edges e ON e.sk = w.kind AND e.si = w.id
    WHERE w.depth < %(depth)s
)
SELECT c.region, COUNT(DISTINCT w.id) AS accounts
FROM walk w
JOIN accounts a  ON w.kind = 'Account' AND a.account_id = w.id
JOIN customers c ON c.customer_id = a.customer_id
WHERE w.id <> %(account)s
GROUP BY c.region
ORDER BY c.region;
```

Q4 Cypher

```
MATCH (a:Account {account_id: $account})
MATCH (a)-[:OWNS|USED_DEVICE*1..<depth>]-(b:Account)
WHERE b <> a
WITH DISTINCT b
MATCH (c:Customer)-[:OWNS]->(b)
RETURN c.region AS region, count(b) AS accounts
ORDER BY region;
```

Q5 SQL

```
SELECT t1.transfer_id AS t1, t2.transfer_id AS t2, t3.transfer_id AS t3,
       t3.receiver_id AS final_receiver
FROM transfers t1
JOIN transfers t2 ON t2.sender_id = t1.receiver_id
                 AND t2.occurred_at >  t1.occurred_at
                 AND t2.occurred_at <= t1.occurred_at + 600
JOIN transfers t3 ON t3.sender_id = t2.receiver_id
                 AND t3.occurred_at >  t2.occurred_at
                 AND t3.occurred_at <= t2.occurred_at + 600
WHERE t1.sender_id = %(account)s
  AND t1.occurred_at >= %(start)s AND t1.occurred_at < %(end)s
  AND 90::bigint  * t1.amount_rwf <= 100::bigint * t2.amount_rwf
  AND 100::bigint * t2.amount_rwf <= 105::bigint * t1.amount_rwf
  AND 90::bigint  * t2.amount_rwf <= 100::bigint * t3.amount_rwf
  AND 100::bigint * t3.amount_rwf <= 105::bigint * t2.amount_rwf
  -- four distinct accounts
  AND t1.receiver_id <> t1.sender_id
  AND t2.receiver_id NOT IN (t1.sender_id, t1.receiver_id)
  AND t3.receiver_id NOT IN (t1.sender_id, t1.receiver_id, t2.receiver_id)
ORDER BY t1, t2, t3, final_receiver;
```

Q5 Cypher

```
MATCH (a:Account {account_id: $account})-[t1:TRANSFER]->(b:Account)
WHERE t1.occurred_at >= $start AND t1.occurred_at < $end AND b <> a
MATCH (b)-[t2:TRANSFER]->(c:Account)
WHERE t2.occurred_at > t1.occurred_at AND t2.occurred_at <= t1.occurred_at + 600
  AND 90 * t1.amount_rwf <= 100 * t2.amount_rwf AND 100 * t2.amount_rwf <= 105 * t1.amount_rwf
  AND c <> a AND c <> b
MATCH (c)-[t3:TRANSFER]->(d:Account)
WHERE t3.occurred_at > t2.occurred_at AND t3.occurred_at <= t2.occurred_at + 600
  AND 90 * t2.amount_rwf <= 100 * t3.amount_rwf AND 100 * t3.amount_rwf <= 105 * t2.amount_rwf
  AND d <> a AND d <> b AND d <> c
RETURN t1.transfer_id AS t1, t2.transfer_id AS t2, t3.transfer_id AS t3,
       d.account_id AS final_receiver
ORDER BY t1, t2, t3, final_receiver;
```

Q6 SQL

```
WITH RECURSIVE chain(cur, ids, first_at, last_at, last_amount, len, returned) AS (
    SELECT receiver_id, ARRAY[transfer_id], occurred_at, occurred_at, amount_rwf, 1,
           receiver_id = %(account)s
```

```
    FROM transfers
    WHERE sender_id = %(account)s
      AND occurred_at >= %(start)s AND occurred_at < %(end)s
    UNION ALL
    SELECT t.receiver_id, c.ids || t.transfer_id, c.first_at, t.occurred_at, t.amount_rwf,
           c.len + 1, t.receiver_id = %(account)s
    FROM chain c
    JOIN transfers t ON t.sender_id = c.cur
                    AND t.occurred_at >  c.last_at          -- strictly increasing
                    AND t.occurred_at <  %(end)s            -- inside the hour
                    AND t.occurred_at <= c.first_at + 1800  -- span at most 1,800 s
    WHERE NOT c.returned                                    -- stop at the first return
      AND c.len < 8
      AND 90::bigint  * c.last_amount <= 100::bigint * t.amount_rwf
      AND 100::bigint * t.amount_rwf  <= 105::bigint * c.last_amount
)
SELECT ids AS transfer_ids, len AS transfers
FROM chain
WHERE returned AND len >= 3
ORDER BY ids;
```

Q6 Cypher

```
MATCH (a:Account {account_id: $account})
MATCH p = (a)-[ts:TRANSFER*3..8]->(a)
WHERE all(t IN ts WHERE t.occurred_at >= $start AND t.occurred_at < $end)
  AND ts[-1].occurred_at - ts[0].occurred_at <= 1800
  AND all(i IN range(1, size(ts) - 1) WHERE
          ts[i].occurred_at > ts[i - 1].occurred_at
          AND 90 * ts[i - 1].amount_rwf <= 100 * ts[i].amount_rwf
          AND 100 * ts[i].amount_rwf <= 105 * ts[i - 1].amount_rwf)
  AND none(n IN nodes(p)[1..-1] WHERE n = a)
RETURN [t IN ts | t.transfer_id] AS transfer_ids, size(ts) AS transfers
ORDER BY transfer_ids;
```

Q7 baseline SQL

```
WITH RECURSIVE bfs(acct, depth) AS (
    SELECT account_id, 0 FROM accounts
    WHERE account_id = %(source)s AND role = 'person'
    UNION
    SELECT t.receiver_id, b.depth + 1
    FROM bfs b
    JOIN transfers t ON t.sender_id = b.acct
    JOIN accounts  r ON r.account_id = t.receiver_id
    WHERE b.depth < %(bound)s
      AND r.role = 'person'          -- restriction applied during the search
)
SELECT MIN(depth) AS min_length FROM bfs WHERE acct = %(dest)s;
```

Q7 baseline Cypher

```
MATCH (s:Account {account_id: $source}), (t:Account {account_id: $dest})
MATCH p = shortestPath((s)-[:TRANSFER*1..<bound>]->(t))
WHERE all(n IN nodes(p) WHERE n.role = 'person')
RETURN length(p) AS min_length;
```

Q7 improved (section 6.3) SQL

```
WITH RECURSIVE fwd(acct, depth) AS (
    SELECT account_id, 0 FROM accounts
    WHERE account_id = %(source)s AND role = 'person'
    UNION
    SELECT n.receiver_id, f.depth + 1
    FROM fwd f
    CROSS JOIN LATERAL (SELECT DISTINCT t.receiver_id FROM transfers t
                        WHERE t.sender_id = f.acct) n
    JOIN accounts r ON r.account_id = n.receiver_id AND r.role = 'person'
    WHERE f.depth < (%(bound)s + 1) / 2
), bwd(acct, depth) AS (
```

```
    SELECT account_id, 0 FROM accounts
    WHERE account_id = %(dest)s AND role = 'person'
    UNION
    SELECT n.sender_id, b.depth + 1
    FROM bwd b
    CROSS JOIN LATERAL (SELECT DISTINCT t.sender_id FROM transfers t
                        WHERE t.receiver_id = b.acct) n
    JOIN accounts s ON s.account_id = n.sender_id AND s.role = 'person'
    WHERE b.depth < %(bound)s / 2
), f AS (SELECT acct, MIN(depth) AS d FROM fwd GROUP BY acct),
   b AS (SELECT acct, MIN(depth) AS d FROM bwd GROUP BY acct)
SELECT MIN(f.d + b.d) AS min_length
FROM f JOIN b USING (acct)
WHERE f.d + b.d <= %(bound)s;
```

Q7 improved (section 6.3) Cypher

```
(same Cypher as above)
```